

P3214R0

2024-04 Library Evolution Poll Outcomes

Published Proposal, 2024-05-22

Authors:

[Inbal Levi - Library Evolution Chair](#) (MPGC Services LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Ben Craig - Library Evolution Assistant Chair](#) (Raven)

[Billy Baker - Library Evolution Incubator Chair](#) (NVIDIA)

[Nevin Liber - Library Evolution Incubator Assistant Chair and Admin Chair](#) (Argonne National Laboratory)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

- 3.1 Poll 1: Send "[P2985R0] A type trait for detecting virtual base classes" to Library Working Group for C++26.
- 3.2 Poll 2: Send "[P2019R5] Thread Attributes" to Library Working Group for C++26.
- 3.3 Poll 3: Apply changes in "[P2855R1] Member customization points for Senders and Receivers" to P2300, and send the revised paper to Library Working Group for C++26.
- 3.4 Poll 4: Send "[P3168R1] Give std::optional range support" to Library Working Group for C++26.

- 3.5 Poll 5: Send "[P2075R5] Philox as an extension of the C++ RNG engines" to Library Working Group for C++26.
- 3.6 Poll 6: Send "[P2927R2] Observing exceptions stored in exception_ptr" to Library Working Group for C++26.
- 3.7 Poll 7: Send "[P2997R1] Removing the common reference requirement from the indirectly invocable concepts" to Library Working Group for C++26.
- 3.8 Poll 8: Send "[P2389R1] dextents Index Type Parameter" to Library Working Group for C++26.
- 3.9 Poll 9: Apply the policy suggested in "[P3201R1] LEWG nodiscard policy" to SD-9.

References

Informative References

§ 1. Introduction

In 2024-04, the C++ Library Evolution group conducted a series of electronic decision polls [\[P3213R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 22 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the papers' authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send "[P2985R0] A type trait for detecting virtual base classes" to Library Working Group for C++26.	9	8	3	0	0	Strong Consensus in favor
Poll 2: Send "[P2019R5] Thread Attributes" to Library Working Group for C++26.	9	5	2	0	4	Weak consensus in favor

Poll 3: Apply changes in "[P2855R1] Member customization points for Senders and Receivers" to P2300, and send the revised paper to Library Working Group for C++26.	11	2	2	0	0	Strong Consensus in favor
Poll 4: Send "[P3168R1] Give std::optional range support" to Library Working Group for C++26.	10	3	1	2	3	Weak consensus in favor
Poll 5: Send "[P2075R5] Philox as an extension of the C++ RNG engines" to Library Working Group for C++26.	5	7	1	2	0	Consensus in favor
Poll 6: Send "[P2927R2] Observing exceptions stored in exception_ptr" to Library Working Group for C++26.	9	6	2	0	2	Consensus in favor
Poll 7: Send "[P2997R1] Removing the common reference requirement from the indirectly invocable concepts" to Library Working Group for C++26.	7	6	2	0	0	Strong Consensus in favor
Poll 8: Send "[P2389R1] dextents Index Type Parameter" to Library Working Group for C++26.	6	5	1	2	1	Consensus in favor
Poll 9: Apply the policy suggested in "[P3201R1] LEWG nodiscard policy" to SD-9.	15	3	1	1	0	Consensus in favor

All the polls have consensus in favor, and all but P3201R1 (which apply to SD-9) and P2855R1 (which apply to P2300) will be forwarded to LWG.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[P2985R0] A type trait for detecting virtual base classes" to Library Working Group for C++26.

With multiple instances of prior art and an "expert-only" implementation, it seems worth standardizing.

— Strongly Favor

I appreciate the paper's extended and detailed discussion of use cases. The move-converting constructor from a `weak_ptr<Y>` to `weak_ptr<T>` is particularly motivating. Given that the conversion does not syntactically appear to involve observing the `shared_ptr`, it might surprise many users for this to involve `lock()`, with its possibly expensive thread synchronization. This paper fixes that for common cases. I also appreciate the paper's careful discussion of the choice to behave differently than prior art, but in alignment with the core language definition of virtual base class.

— Strongly Favor

This trait is useful, and I agree with the author that we shouldn't wait for "Reflection" to land, as this will only be one of many traits, all of which we would need to re-consider.

— Weakly Favor

This is very niche and I don't see much practical impact. However, this is a minor addition that fits well into the existing library.

— Neutral

§ 3.2. Poll 2: Send "[P2019R5] Thread Attributes" to Library Working Group for C++26.

This might make it possible to actually use the standard library for creating threads.

— Strongly Favor

Most users that create threads at least want the ability to name threads, as this helps debugging. Many users also want to control threads' stack size. It would be unfortunate if users found themselves unable to use `std::thread` or `std::jthread` just because they couldn't pass these common attributes to threads. The paper surveys many existing operating systems for their ability to create or query threads with the proposed attributes. The resulting design is a zero-overhead design, because it does not require storing attributes separately if the operating system's thread does not have a query mechanism. Using a named struct with named fields, as in an alternate proposal, would give users incorrect expectations about ABI stability. This paper's approach also makes adding new attribute types straightforward. It would be excellent if there were a way to ask the operating system if an attribute were ignored or incorrect (e.g., if the string were too long), but that's not a thing we can expect to be portable. I understand that attributes are hints and that implementations can treat them as best-effort only.

— Strongly Favor

I think the paper proposes the best solution compared to other "Thread Attributes" proposal that I've seen.

— Strongly Favor

I like the functionality that this paper provides, and the design is relatively clean and usable. The motivation for making the attributes non-movable to avoid copies/dangling-references seems weak. You can still construct a local variable of type `thread::name_hint` from a temporary `std::string` and then pass that local variable to a thread constructor and end up with a dangling reference. I would also have liked to see explored a design based on the `queryable` concept from P2300, but am not going to hold up the paper based on this.

— Weakly Favor

My preference would be to have a general, extensible facility for a bag-o-properties that can be used for thread attributes or p2300's execution environments and sender attributes and many other places besides, but i won't hold this feature up for that.

— Weakly Favor

I would have appreciated more interoperability: if one already has a threading API, they cannot use the newly introduced hints as there is no way to 1) identify that a given parameter is an hint, and 2) to extract the value of that parameter.

— Weakly Favor

An unnecessarily clever keyword-argument API is incoherent with the standard library and presents yet another obstacle for those considering C++ adoption.

— Strongly Against

Feels like a non C++ approach; we should have a single attributes class

— Strongly Against

§ 3.3. Poll 3: Apply changes in "[P2855R1] Member customization points for Senders and Receivers" to P2300, and send the revised paper to Library Working Group for C++26.

`tag_invoke` was very clever but it's impact on compile times not acceptable. This is a simple, sufficient solution until we can fix CPO in the language.

— Strongly Favor

Member functions are much more readable and teachable than `tag_invoke`. It's not that `tag_invoke` is bad, it's that having an actual language feature for customization points would be a much better solution. EWG is heavily occupied with other high-priority proposals aiming for C++26. Thus, it's unlikely WG21 would have time to finish the proposed customization points language feature. We also very much would like WG21 to approve P2300 for C++26. Member functions are not hard to use and are not unreasonable for the P2300 use case, so I think it's fine for P2300 to adopt the member functions approach. Nevertheless, I still think EWG should pursue the language feature.

— Strongly Favor

Although this is an improvement on the status quo, it's regrettable that we are continuing the trend of adding opt-in concepts. Concepts should be duck typed.

— Weakly Favor

§ 3.4. Poll 4: Send "[P3168R1] Give `std::optional` range support" to Library Working Group for C++26.

Although I still instinctively feel that this is somewhat unintuitive, I've found myself adequately convinced that it's the most elegant way to address `std::ranges` pipelines that need to handle optionality. It also avoids us having to introduce the redundant `views::maybe` type.

— Strongly Favor

A range of 0 or 1 shows up very often in algorithm work, and the standard needs one. While a dedicated view might be useful and not add to the complexity of `std::optional`, this is a straightforward solution that is generally useful.

— Strongly Favor

This seems to be the conclusion of all the LEWG discussion around various proposals. I agree that this direction minimizes "new things to learn." I also agree that the iterator type should be implementation-defined rather than `T*` or `T* const` (in fact, I would prefer that it be defined `_not_` to be a pointer and that it not have public conversions to pointer, but I accept making the type implementation-defined).

— Strongly Favor

Provided this is actually necessary, it is a better solution than introducing a new word.

— Weakly Favor

It's useful to be able to treat `optional` as a range, and this paper is strictly superior to the design it replaces (having a duplicate `std::optional` that is only usable as a range, mostly), but I wonder if simply having an adaptor is sufficient. Not opposed, just maybe not actively for.

— Neutral

This seems to add more and more tangential features to optional. I wasn't in the discussion, but the minutes suggest a fair number of similar counter arguments (e.g. should `_every_` object be a range-of-one?) with no discussion reflected in the minutes. I don't understand the positive poll outcome from the minutes.

— Weakly Against

This is mixing concerns and will ultimately lead to ambiguities, all to save a few keystrokes (given an optional both `for(auto x : *opt)` and `for(auto x : opt)` are valid syntaxes with different aspect.) By the logic of this paper should any object be convertible to `view::single`?

— Strongly Against

I feel like with this proposal we give a wrong impression of the `std::optional` semantics because `std::optional` is not a range and we should not pretend it is. With the given proposal we could ask ourselves why is not `std::variant` a range? It could potentially return active element as well on the iterator dereference. Basically we might create a precedence where each type wrapper with special properties (like optional or variant or something else) would be considered as range, which to me is not a good idea.

— Strongly Against

§ 3.5. Poll 5: Send "[P2075R5] Philox as an extension of the C++ RNG engines" to Library Working Group for C++26.

This proposal shows the strength of the C++ Standard Library's random number generator design, in that it's straightforward to add new generators. The paper also highlights that the Philox engine has several implementations by hardware vendors. The authors carefully explain and justify the features of `philox_engine` that differ from the existing Standard engines. In particular, I very much appreciate the presentation of alternative interfaces for `set_counter`, and the explanation why `get_counter` is not included.

— Strongly Favor

This random number engine is well-motivated and has extensive implementation experience. The state of the art for random number generation has advanced since we were standardizing the generators introduced in C++11.

— Strongly Favor

Yes. And let's get a trivial, lightweight RNG like PCG in there for the use-cases where the user doesn't need such statistical-stringent generation and doesn't want to rely on SIMD availability. More info (from <https://github.com/colesbury/benchmark-rngs>): Time to get 1000000000 philox randoms = 0.271853s Time to get 1000000000 philox_simd randoms = 0.117607s Time to get 1000000000 xoshiro256** randoms = 0.0561525s Time to get 1000000000 pcg64 randoms = 0.0990407s Time to get 1000000000 std::mt19937 randoms = 0.195286s

— Strongly Favor

Keeping up with the current state of RNGs was a promise C++ made, perhaps by accident. Making sure the standard library is useful is useful. I'm weakly in favor only because RNGs are not my domain.

— Weakly Favor

Been verified with ARM, AMD, NVIDIA, and Intel, and seems like a solid feature with field experience.

— Weakly Favor

This proposal needs more implementation and user experience. Moreover, none of the linked-to implementations match the specification. This is not standards-level quality.

— Weakly Against

§ 3.6. Poll 6: Send "[P2927R2] Observing exceptions stored in exception_ptr" to Library Working Group for C++26.

Simplification after Kona 2023 was not discussed.

— Strongly Favor

I've wanted a facility like this a few times. It always seemed silly to have an `exception_ptr` -- the "exception equivalent of `any`" -- in hand, and yet have to do a try-catch dance to get to the exception object. This proposal fixes that. The authors did a great job responding to LEWG feedback on how to make `exception_ptr_cast` more usable (so users don't have to worry about `const` qualification) while at the same time avoiding unimplementable cases.

— Strongly Favor

This paper solves an important problem, but I really wish we had followed the precedent set by `any_cast` and `std::variant::get_if` and required this function to take its parameter by pointer.

— Weakly Favor

`exception_ptr_cast` is the wrong name (too verbose). It should be `exception_cast` or `try_cast`.

— Neutral

§ 3.7. Poll 7: Send "[P2997R1] Removing the common reference requirement from the indirectly invocable concepts" to Library Working Group for C++26.

Coroutines are leading to generators. Generators lead to situations like the ones this paper fixes. Even Python generators are single-pass, so it's reasonable to give "ordinary" C++ users single-pass iterators that come from things like coroutines. The authors make a strong argument that this would only affect usability (the requirements on users' callables) and not the algorithms themselves. It's enough motivation just to improve the error messages. The GCC 13.2 message in Section 2's first example would be difficult to understand or explain for most users.

— Strongly Favor

This fixes a clear bug in these concepts and is a textbook example of a situation where we should make an exception to our general guideline of not modifying concepts after standardization.

— Strongly Favor

I think the behavior that this paper proposes is correct because the issue that is described in the proposal looks really weird from the users standpoint.

— Weakly Favor

Solid arguments, I agree that we shouldn't "propagate" the Feature test macro promoted.

— Weakly Favor

This appears to be a minor bugfix.

— Neutral

§ 3.8. Poll 8: Send "[P2389R1] `dextents` Index Type Parameter" to Library Working Group for C++26.

Users sometimes need to name `mdspan` types. They can't always rely on CTAD. As a result, users need to name `extents` specializations. `dextents` was designed to make the common case of all dynamic extents easy and concise. Later changes to `extents` took away some of that conciseness. This proposal brings back some of that. It does so without breaking backwards compatibility, by introducing a new name `dims`. Given that `dims` has an optional `IndexType` template parameter, one could argue that `dextents` is no longer needed and should be deprecated. On the other hand, it's harmless to keep around. We could always deprecate it in a separate paper.

— Strongly Favor

The status quo of `dextents` was clearly broken; since we didn't want to do a source-breaking change, we needed to introduce an alternative, and the specification for `dims` here is the right one.

— Strongly Favor

Reducing the syntactic noise around declaring `mdspan` is beneficial.

— Weakly Favor

Abbreviation and clarity is good, but I would be wary in future of providing such things for the sole purpose of code reduction. Where possible, reducing the count of necessary items to memorize in the C++ library should take priority over code reduction for expert users. I also oppose `_t`-style traits for this reason.

— Weakly Against

The proposed name seems to potentially confuse "dimension" and "rank". I realize that "dims" should be read as "number of dimensions", so that "dims<3>" means "has 3 ‘dimension’ values", but this seems subtle and easy to confuse with "has 3 dimensions" or "is 3-dimensional". Something like "std::ndims" or "std::dyn_rank" would avoid such issues.

— Strongly Against

§ 3.9. Poll 9: Apply the policy suggested in "[P3201R1] LEWG nodiscard policy" to SD-9.

I think this is the best possible outcome for nodiscard policy

— Strongly Favor

Implementers bring a lot more information and better judgement to bear on the question of whether any particular interface should be marked `[noexcept]` than LEWG does.

— Strongly Favor

Spending time on providing hints to implementors that they neither desire or necessarily respect is a waste of precious committee time. The proliferation of `[nodiscard]` in library implementation indicates that implementors are doing the right thing without committee input or requirement.

— Strongly Favor

While I understand that LEWG doesn't want to spend time in nodiscard discussions, users still look at the Standard as a source of inspiration, precedents, and good practices. Having a policy that we don't use [nodiscard] in the Standard text may start raise questions about the usefulness of nodiscard itself.

— Strongly Favor

"Has a spurious impact" is still ungrammatical/unclear. But the policy is correct.

— Strongly Favor

Make less work for us. Yay!

— Strongly Favor

I don't understand how this is better than Wakely's previous paper which at least attempts to provide categories for where we should use [nodiscard], but it's clearly better than attempting to enumerate those places like some alternate papers, and if this gets us to stop talking about it all the time - then sure.

— Weakly Favor

While I like the intent of making it completely QoI I do think a recommendation of what should be reasonably marked no discard in some official document (could be SD-9, the standard, etc) would help ensuring the quality of implementations is aligned with the expectations of their users.

— Weakly Against

Editor's note: The policy contains a link to the paper, which contains the information on suggested locations where [[nodiscard]] should appear, so the information remains available (even if not as part of the policy itself).

§ References

§ Informative References

[P3213R0]

Inbal Levi; et al. *2024-04 Library Evolution Polls*. 15 May 2024. URL:
<https://wg21.link/p3213r0>